Differentiable Zero-One Loss via Hypersimplex Projections

Camilo Gomez[1]*, Pengyang Wang[2], and Liansheng Tang[1]

[1] School of Data, Mathematical, and Statistical Sciences, University of Central Florida, Orlando, USA
camilo.gomez@ucf.edu, liansheng.tang@ucf.edu
[2] Department of CIS, University of Macau, Macao, China
pywang@um.edu.mo

Abstract. Recent advances in machine learning have emphasized the integration of structured optimization components into end-to-end differentiable models, enabling richer inductive biases and tighter alignment with task-specific objectives. In this work, we introduce a novel differentiable approximation to the zero–one loss—long considered the gold standard for classification performance, yet incompatible with gradient-based optimization due to its non-differentiability. Our method constructs a smooth, order-preserving projection onto the (n, k)-dimensional hypersimplex through a constrained optimization framework, leading to a new operator we term Soft-Binary-Argmax. After deriving its mathematical properties, we show how its Jacobian can be efficiently computed and integrated into binary and multiclass learning systems. Empirically, our approach achieves significant improvements in generalization under large-batch training by imposing geometric consistency constraints on the output logits, thereby narrowing the performance gap traditionally observed in large-batch training. Our code is available here https://github.com/camilog04/Differentiable-Zero-One-Loss-via-Hypersimplex-Projections.

Keywords: Differentiable Optimization in Deep Learning · Differentiable programming · Large-batch generalization.

1 Introduction

Recent developments in machine learning have demonstrated that optimization procedures can be used as fundamental components within end-to-end differentiable systems [1, 6, 10, 11]. Rather than relying solely on traditional neural network layers, these approaches incorporate more structured, often nontrivial computations—such as constrained optimization—directly into the learning pipeline. These components usually have structural or computational properties that have been proven useful in downstream tasks. For instance, Sparsemax, a differentiable projection onto the simplex, produces sparse posterior distributions that are effective as attention mechanisms [17]. Similarly, Csoftmax,

* Corresponding author

a projection onto the budget polytope, has demonstrated utility in sequence tagging [18]. This reflects a growing shift toward viewing learning systems as differentiable computational frameworks that blend elements of traditional statistical modeling with algorithmic computation [5]. In this paper, we focus on crafting a differentiable, order-preserving projection into the n, k-dimensional hypersimplex [7]–a well-studied combinatorics polytope–in composition with a squared loss to generate a close approximation to the zero-one, misclassification loss compatible with modern large-scale differentiable systems.

From a theoretical perspective in machine learning, the goal is to minimize the expected value of a task-specific loss function over a data distribution. For classification, the most natural choice is the zero-one loss, which directly measures misclassification error. However, zero-one loss is non-differentiable and discontinuous, as it depends on a hard threshold decision—yielding gradients of zero almost everywhere and rendering it incompatible with gradient-based optimization. To enable tractable training, modern approaches rely on surrogate losses (e.g., cross-entropy, hinge loss) that are smooth and differentiable [3]. These surrogates serve as proxies that approximate the zero-one loss while facilitating efficient optimization. Despite their practicality, such surrogates often exhibit a mismatch with the true evaluation metric, especially under large-batch regimes. This degradation in performance with large batch sizes is the so-called generalization gap [19], leading to growing interest in tighter, more faithful approximations to the zero-one loss, under the hypothesis that closer surrogates yield better generalization.

This work introduces a fully differentiable approximation to the zero–one loss, featuring an efficient forward pass with complexity $\mathcal{O}(n \log n)$ and a backward pass with $\mathcal{O}(n)$ complexity. This is achieved through our novel differentiable projection layer, **Soft-Binary-Argmax@k**. Rather than treating the output scores as independent, our layer explicitly enforces that the largest k logits correspond to the predicted positive classes. This design ensures that small perturbations in the input produce coherent, structurally consistent adjustments in the output, making the Jacobian of the transformation inherently *positionally aware* with respect to the most confident predictions. The benefits of this approach are twofold: it allows binary classifiers to express multiple positive outcomes within a single forward pass, and it extends naturally to the multiclass setting by applying the same projection principle across one-hot encoded class dimensions. Importantly, the geometric constraints imposed on the output logits act as a form of regularization that mitigates the generalization degradation typically observed under large-batch training, enabling stable optimization and improved predictive performance.

More precisely, our contributions and novelty can be summarized as follows:

1. We introduce a differentiable projection layer—a smooth thresholding operator realized via projection onto the interior of the n, k-dimensional hypersimplex—termed **Soft-Binary-Argmax@k**. It provides a differentiable relaxation of the binary *argmax*, reduces to isotonic regression, and enables efficient forward and backward computation on both CPU and GPU.

2. We propose a smooth, almost-everywhere differentiable loss function for binary classification, the **HyperSimplex Loss**, and derive its mathematical properties. The loss couples the mean squared error with our projection layer and extends naturally to multiclass.
3. Through rigorous experimentation, we provide empirical evidence that the proposed loss mitigates the generalization gap and improves performance across multiple classification benchmark datasets.

2 Related work

2.1 Differentiable optimization-based ML

Optimization-based modeling integrates structure and constraints into machine learning architectures by embedding parameterized *argmin*/*argmax* operations as differentiable layers. Such layers are often formulated as convex, constrained programs, with differentiability achieved via the implicit function theorem applied to the KKT conditions [11]. Parallel work has explored differentiable optimization for order-constrained or monotonic outputs, including differentiable isotonic regression operators for smooth, order-aware learning [6]. To the best of our knowledge, no prior work has introduced a differentiable Euclidean projection onto the (n, k)-dimensional hypersimplex—computed via the Pool Adjacent Violators (PAV) algorithm—as a learnable layer. Our approach fills this gap, providing an efficient and theoretically grounded formulation for integrating hypersimplex projections into modern differentiable systems.

2.2 Generalization gap

A well-known challenge in modern deep learning is the *generalization gap*, where models trained with large batch sizes achieve low training loss but exhibit degraded test performance. This phenomenon has been widely observed in neural networks, as large batches tend to converge to sharp minima that generalize poorly compared to the flatter solutions found by small-batch training [15]. Subsequent work has explored remedies such as adaptive learning rate schedules and warmup strategies, noise injection and regularization [12], and stochastic weight averaging [14] to mitigate this effect. However, to the best of our knowledge, our work is the first to address the generalization gap *through loss function design*, introducing a principled framework that directly links the geometry of the loss landscape to generalization behavior.

3 Preliminaries

In supervised multiclass classification, we are given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, where each input $x_i \in \mathcal{X} \subset \mathbb{R}^d$ is associated with a categorical label $y_i \in \{1, \ldots, C\}$ among C possible classes. Let $f : \mathcal{X} \to \mathbb{R}^C$ denote a prediction

function producing a score vector $f(x_i) = (f_1(x_i), \ldots, f_C(x_i))^\top$, where each component $f_c(x_i)$ reflects the model's confidence for class c.

The learning objective is to minimize the *multiclass zero–one loss*, which measures the fraction of misclassified samples:

$$\mathcal{L}_{0/1}(f) = \frac{1}{n}\sum_{i=1}^{n} \mathbb{I}[\,\hat{y}_i \neq y_i\,], \qquad \hat{y}_i = \arg\max_{c\in\{1,\ldots,C\}} f_c(x_i). \tag{1}$$

While $\mathcal{L}_{0/1}$ directly quantifies classification accuracy, it is discontinuous and non-differentiable, making it unsuitable for gradient-based optimization.

To obtain a differentiable surrogate, convex losses are commonly employed. Common loss functions in machine learning—such as squared loss, hinge loss, and logistic loss—are convex approximations of the true 0–1 misclassification loss [3]. Among these, the squared loss provides the closest approximation to the 0–1 loss on the interval $(0, 1)$, making it a natural foundation for our formulation. The *multiclass squared loss* penalizes deviations between predicted scores and the corresponding one-hot target encodings, summing over all classes:

$$\mathcal{L}_{\mathrm{sq}}(f) = \frac{1}{n}\sum_{i=1}^{n}\sum_{c=1}^{C} \big(f_c(x_i) - \mathbb{I}[y_i = c]\big)^2. \tag{2}$$

Equivalently, in matrix form, $\mathcal{L}_{\mathrm{sq}}(f) = \frac{1}{n}\|F(X) - Y\|_F^2$, where $F(X) \in \mathbb{R}^{n\times C}$ collects the model outputs and Y is the one-hot label matrix. While this smooth, convex loss provides analytic gradients and serves as a tractable approximation to $\mathcal{L}_{0/1}$, it also suffers from a major drawback: it imposes a quadratic penalty on extreme predicted values, leading to sensitivity to outliers [8,13]. This limitation motivates our projection-based formulation introduced next, which preserves smoothness while constraining outputs within a geometrically consistent region.

4 Methodology

4.1 Overview

This work begins by formulating the problem in the binary classification setting, where the objective is to distinguish between positive and negative outcomes. The same geometric principles, however, extend naturally to the multiclass setting, as shown in later sections. Common loss functions in machine learning—such as squared loss, hinge loss, and logistic loss—are convex surrogates of the true 0–1 misclassification loss [3]. However, the true 0–1 loss minimizer lies at one of the vertices of the (n, k)-dimensional hypersimplex, as it satisfies two key properties:

1. Its entries are binary, i.e., each component of $f(\mathbf{X})$ takes a value in $\{0, 1\}$.
2. For any given sample $\mathbf{y}$ drawn from the distribution of Y, a perfect prediction vector should contain exactly k positive entries, matching the number of positives in $\mathbf{y}$; that is, $\|f(\mathbf{X})\|_1 = \|\mathbf{y}\|_1 = k$.

Motivated by the geometry of the optimal solution, we now introduce a series of relaxations that make the learning problem tractable. First, we relax the binary constraint and allow $f(\mathbf{X})$ to take real values in $\mathbb{R}^n$, while encouraging sparsity—pushing predictions as close as possible to 0 or 1. To balance smoothness with structural fidelity, we compose our differentiable projection operator, the **soft-binary-argmax@k**, which produces sparse and nearly binary outputs, with the squared loss, which ensures smooth optimization and stability. This composition yields a surrogate objective that remains differentiable while closely aligning with the discrete geometry of the hypersimplex.

The remainder of this section is organized as follows. We begin by establishing the connection between binary-argmax@k and thresholding. Next, we formulate binary-argmax@k as a projection onto the n, k hypersimplex, from which we derive its continuous relaxation, soft-binary-argmax@k, and analyze its key properties. Finally, we combine the soft-binary-argmax@k with a squared loss to define the HyperSimplex loss, and demonstrate its effectiveness in generalization for large batch sizes.

4.2 Thresholding and the Binary-Argmax@k

This section establishes an intuitive connection between a real-valued vector $\mathbf{x} \in \mathbb{R}^n$ and its binary counterpart in $\{0,1\}^n$. A common discretization method is *thresholding*, where entries exceeding a fixed boundary (typically 0.5) are set to 1, and the rest to 0. Although simple, this approach ignores relative ordering and offers no control over the number of positive components.

A more structured alternative is the binary-argmax@k operator, denoted r_k, which assigns 1 to the k largest entries of $\mathbf{x}$ and 0 to the remaining $n-k$. Here, the threshold is adaptively defined by the k-th largest value of $\mathbf{x}$, reducing to standard thresholding when $k = \lceil n/2 \rceil$, where the threshold equals the empirical median of the logits.

Formally, let $\mathbf{x} \in \mathbb{R}^n$ be a vector of scores and $k \in \{1, \ldots, n\}$. We define the binary-argmax@k operator as

$$r_k(\mathbf{x}) = \mathbb{I}(x_i \geq T_k(\mathbf{x}))\,, \quad T_k(\mathbf{x}) = k\text{-th largest value of } \mathbf{x}. \tag{3}$$

This rule ensures exactly k components of $\mathbf{x}$ are set to 1, enforcing the constraint

$$\|r_k(\mathbf{x})\|_1 = k, \quad r_k(\mathbf{x}) \in \{0,1\}^n. \tag{4}$$

The binary-argmax@k mapping does not provide useful derivatives, hindering gradient-based optimization. To address this, we formulate it as a linear optimization problem over the (n,k)-dimensional hypersimplex Δ_k^n and introduce a Euclidean regularization term with a temperature parameter, yielding a smooth relaxation—the soft-binary-argmax@k. This differentiable formulation preserves the hypersimplex geometry while providing informative gradients for end-to-end learning.

4.3 Binary-Argmax@k: Euclidean Projections onto the Hypersimplex

The Euclidean projection onto the (n, k)-dimensional hypersimplex can be expressed as the solution of a simple regularized linear program:

$$\underset{\mathbf{y}\in\mathbb{R}^n}{\text{argmax}} \ \langle \mathbf{x}, \mathbf{y}\rangle - \|\mathbf{y}\|_2^2 \quad \text{s.t.} \quad \mathbf{1}^\top \mathbf{y} = k, \quad 0 \le \mathbf{y} \le 1. \tag{5}$$

The first term encourages alignment with the input vector $\mathbf{x}$, while the quadratic regularization term enforces proximity to the origin, thereby inducing a balance between sparsity and fidelity. The affine constraint $\mathbf{1}^\top \mathbf{y} = k$ fixes the ℓ_1 mass of $\mathbf{y}$, ensuring exactly k active components, while the box constraint $0 \le \mathbf{y} \le 1$ confines the solution to the hypercube $[0, 1]^n$.

The feasible region defined by these two constraints is precisely the (n, k)-dimensional hypersimplex:

$$\Delta_k^n = \left\{ \mathbf{y} \in [0, 1]^n \ \middle| \ \sum_{i=1}^{n} y_i = k \right\}. \tag{6}$$

Hence, the optimization problem in (5) is equivalent to the Euclidean projection over the hypersimplex:

$$\Pi_{\Delta_k^n}(\mathbf{x}) = \underset{\mathbf{y}\in\Delta_k^n}{\text{argmin}} \ \|\mathbf{x} - \mathbf{y}\|_2^2. \tag{7}$$

Since Δ_k^n is convex and compact, this problem admits a unique solution. Geometrically, $\Pi_{\Delta_k^n}(\mathbf{x})$ corresponds to the point within Δ_k^n that lies closest to $\mathbf{x}$ in Euclidean distance, and algebraically, it coincides with the binary vector that activates the k largest components of $\mathbf{x}$, the binary-argmax@k.

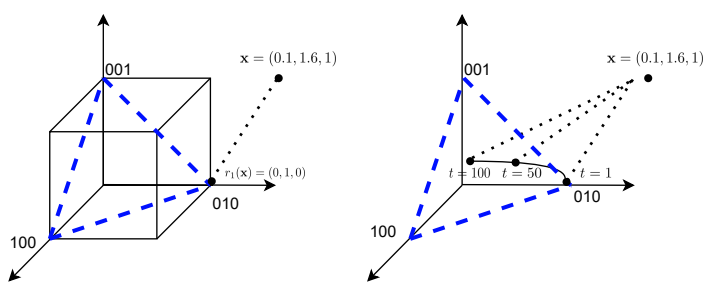


Fig. 1. Binary-argmax@k of a point $\mathbf{x} = (0.1, 1.6, 1)$ into the exterior of the Hypersimplex (left). At $k = 1$, the solution is $(0, 1, 0)$. Introducing temperature to the program yields an interior solution (right), i.e., the soft-binary-argmax@1. In $\mathbb{R}^3$ different k values yield points on a standard simplex, but in higher dimensions yields a point on the hypersimplex. In $\mathbb{R}^4$ with $k = 2$, the solution lies on an octahedron [2].

4.4 Soft-Binary-Argmax@k: A Differentiable Approximation

The projection $\Pi_{\Delta_k^n}(\mathbf{x})$ provides a geometric mapping from a continuous vector $\mathbf{x}$ to its structured binary counterpart, but it remains piecewise constant and thus non-differentiable. Small perturbations in $\mathbf{x}$ can abruptly change the identity of the top-k elements, yielding discontinuities and zero gradients almost everywhere. Consequently, the hard binary-argmax@k operator is incompatible with gradient-based optimization.

Temperature-Scaled Relaxation. To obtain a smooth approximation, we introduce a temperature parameter $\tau > 0$ that scales the regularization strength in the projection objective:

$$\operatorname*{argmin}_{\mathbf{y}\in\Delta_k^n} \left(\tau\|\mathbf{y}\|_2^2 - 2\langle\mathbf{x},\mathbf{y}\rangle\right) = \operatorname*{argmin}_{\mathbf{y}\in\Delta_k^n} \left(\|\mathbf{y}\|_2^2 - 2\left\langle\tfrac{\mathbf{x}}{\tau},\mathbf{y}\right\rangle\right) \tag{8}$$

leading to the compact expression

$$\Pi_\tau(\mathbf{x}) = \operatorname*{argmin}_{\mathbf{y}\in\Delta_k^n} \left\|\mathbf{y} - \tfrac{\mathbf{x}}{\tau}\right\|_2^2 = \Pi_{\Delta_k^n}\left(\tfrac{\mathbf{x}}{\tau}\right) \tag{9}$$

As $\tau \to 0$, the operator recovers the discontinuous hard projection, while larger τ values yield smoother outputs closer to the hypersimplex—defining the soft-Binary-Argmax@k operator.

Proposition 1 (Differentiability a.e). *Fix $k \in \{1,\dots,n\}$ and $\tau > 0$. The mapping*

$$F_\tau : \mathbb{R}^n \to \Delta_k^n, \qquad F_\tau(\mathbf{x}) := \Pi_{\Delta_k^n}\left(\tfrac{\mathbf{x}}{\tau}\right)$$

is $(1/\tau)$-Lipschitz, hence differentiable almost everywhere (a.e.) in $\mathbb{R}^n$.

Proof. The Euclidean projection onto a closed convex set in a Hilbert space is nonexpansive: $\|\Pi_C(\mathbf{u}) - \Pi_C(\mathbf{v})\|_2 \le \|\mathbf{u}-\mathbf{v}\|_2$ for all $\mathbf{u},\mathbf{v}$. With $C = \Delta_k^n$ and $\mathbf{u} = \mathbf{x}/\tau$, $\mathbf{v} = \mathbf{z}/\tau$,

$$\left\|F_\tau(\mathbf{x}) - F_\tau(\mathbf{z})\right\| = \left\|\Pi_{\Delta_k^n}\left(\tfrac{\mathbf{x}}{\tau}\right) - \Pi_{\Delta_k^n}\left(\tfrac{\mathbf{z}}{\tau}\right)\right\| \le \left\|\tfrac{\mathbf{x}}{\tau} - \tfrac{\mathbf{z}}{\tau}\right\| = \tfrac{1}{\tau}\,\|\mathbf{x}-\mathbf{z}\|.$$

Thus F_τ is $(1/\tau)$-Lipschitz. By Rademacher's theorem, every Lipschitz map on $\mathbb{R}^n$ is differentiable a.e., proving the claim.

Proposition 2 (Order preservation). *The projection solution $y_i = \Pi_{\Delta_k^n}\left(\frac{x_i}{\tau}\right)$ is order preserving; that is, if $x_1/\tau \ge x_2/\tau \ge \cdots \ge x_n/\tau$, then the projected coordinates satisfy $y_1 \ge y_2 \ge \cdots \ge y_n$.*

Proof. From the KKT conditions of the Lagrangian associated with (9), stationarity and complementarity yield, for each i,

$$y_i = \operatorname{clip}\left(x_i/\tau - \tfrac{\lambda}{2},\, 0,\, 1\right),$$

where the multiplier λ is uniquely determined to satisfy the equality constraint $\sum_i y_i = k$. Since the mapping $t \mapsto \operatorname{clip}\left(t - \frac{\lambda}{2}, 0, 1\right)$ is monotone nondecreasing in t, it follows that $x_1/\tau \ge \cdots \ge x_n/\tau \Rightarrow y_1 \ge \cdots \ge y_n$. See [9] for details.

Corollary 1 (Computation). *Since the projection is order preserving (Proposition 2), adding a monotonicity constraint does not change the solution. Hence, for any sorted input $\mathbf{x}/\tau$, the projection can be computed via a reduction to isotonic regression:*

$$\Pi(\mathbf{x}/\tau) = \underset{\substack{\mathbf{y}\in[0,1]^n,\\ \mathbf{1}^\top\mathbf{y}=k,\\ y_1\geq\cdots\geq y_n}}{\arg\min} \left\|\tfrac{\mathbf{x}}{\tau} - \mathbf{y}\right\|^2.$$

The feasible set is closed and convex, ensuring a unique and differentiable solution. This reduces to a standard isotonic projection problem, solvable efficiently via the pool-adjacent-violators (PAV) algorithm [4] in $\mathcal{O}(n \log n)$ time.

4.5 The HyperSimplex loss

We now compose our projection operator with the squared loss to define a smooth surrogate for binary classification. The squared loss provides high fidelity to the zero–one objective within $(0, 1)$ but can be dominated by large-magnitude predictions. By composing it with the projection operator $\Pi_{\Delta_n^k}$, we constrain predictions to the hypersimplex, preventing any coordinate from overtaking the loss while preserving the discrete geometry of the solution.

Formally, for $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, define

$$\hat{\mathbf{y}} = \Pi_{\Delta_n^k}\left(\frac{\mathbf{x}}{\tau}\right), \qquad L(\mathbf{x}, \mathbf{y}) = \tfrac{1}{2}\|\hat{\mathbf{y}} - \mathbf{y}\|_2^2,$$

where $\tau > 0$ controls the smoothness of the relaxation. The gradient with respect to $\mathbf{x}$ follows from the chain rule:

$$\nabla_{\mathbf{x}} L(\mathbf{x}, \mathbf{y}) = \frac{1}{\tau}\, J_\Pi\left(\tfrac{\mathbf{x}}{\tau}\right)(\hat{\mathbf{y}} - \mathbf{y}),$$

where J_Π denotes the Jacobian of the projection operator $\Pi_{\Delta_n^k}$. Let $A = \{i : 0 < \hat{y}_i < 1\}$ denote the active coordinates. On this set, the Jacobian acts as

$$J_\Pi = I_{|A|} - \tfrac{1}{|A|}\mathbf{1}\mathbf{1}^\top,$$

yielding the component-wise gradient

$$(\nabla_{\mathbf{x}} L)_i = \begin{cases} \dfrac{1}{\tau}\left[(\hat{y}_i - y_i) - \dfrac{1}{|A|}\sum_{j\in A}(\hat{y}_j - y_j)\right], & i \in A, \\ 0, & i \notin A. \end{cases}$$

At boundary points where some $\hat{y}_i \in \{0, 1\}$, the mapping is only directionally differentiable, and any subgradient consistent with this Jacobian form is valid.

4.6 Extension to Multiclass Classification

The formulation extends naturally to the multiclass setting. For each class $c \in \{1, \ldots, C\}$ with logits $\mathbf{x}^{(c)} \in \mathbb{R}^n$, one-hot target $\mathbf{y}^{(c)}$, and temperature $\tau_c > 0$, we project onto the (n, k_c)-hypersimplex:

$$\mathbf{p}^{(c)} = \Pi_{\Delta_{k_c}^n}\left(\tfrac{\mathbf{x}^{(c)}}{\tau_c}\right).$$

The total loss is

$$\mathcal{L}(X,Y) = \tfrac{1}{2}\sum_{c=1}^{C}\left\|\mathbf{p}^{(c)} - \mathbf{y}^{(c)}\right\|_2^2, \qquad \nabla_{\mathbf{x}^{(c)}}\mathcal{L} = \tfrac{1}{\tau_c} J_{\Pi}\left(\tfrac{\mathbf{x}^{(c)}}{\tau_c}\right)(\mathbf{p}^{(c)} - \mathbf{y}^{(c)}).$$

This provides a smooth, per-class projection framework that preserves hypersimplex structure while remaining fully differentiable. During the learning process each k_c is set to match the expected number of positive responses for class c.

5 Experiments

For our experiments, we evaluate the effectiveness of the proposed HyperSimplex loss in reducing the generalization gap compared to standard classification losses, including Cross-Entropy, Hinge, and Mean Squared Error (MSE, without projection). This setup also serves as an ablation study to isolate the contribution of our projection layer, verifying that incorporating geometric constraints on the output logits yields more consistent performance across batch sizes than the MSE objective alone.

5.1 Datasets

We conduct experiments on two standard image classification benchmarks: CIFAR-10 [16] and Fashion-MNIST [21]. CIFAR-10 consists of 60,000 color images of size 32×32 pixels, split into 50,000 training and 10,000 test samples across 10 object categories. Fashion-MNIST contains 70,000 grayscale images of size 28×28 pixels, divided into 60,000 training and 10,000 test images from 10 clothing categories, serving as a more challenging replacement for the original MNIST dataset.

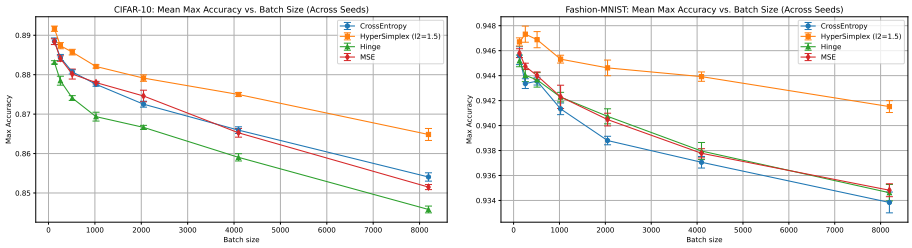
5.2 Experimental Setup

We employed a standard convolutional neural network (CNN) for multiclass image classification, consisting of four convolutional layers, each followed by batch normalization, max pooling, and ReLU activation. The final feature map is flattened and passed through two fully connected layers, with the last layer producing class logits. The datasets were preprocessed using random cropping, horizontal flipping, and per-channel normalization, and randomly split into training and test sets. All experiments were implemented in PyTorch [20] and executed on 32-core AMD Ryzen Threadripper PRO 5975WX CPU with 503 GB of RAM and three NVIDIA RTX 6000 Ada Generation GPUs, each with 48 GB of VRAM.

To ensure statistical robustness, each configuration was trained using five independent random seeds, varying both model initialization and data splits. We evaluated four loss functions—our proposed **HyperSimplex loss** and three widely used baselines: Cross-Entropy, Hinge, and Mean Squared Error (MSE)–across seven batch sizes (128, 256, 512, 1024, 2048, 4096 and 8192) on both

CIFAR-10 and Fashion-MNIST. In total, this resulted in 280 training runs. For each configuration, we recorded the maximum test accuracy achieved per loss function and batch size, and assessed differences against the Cross-Entropy baseline using paired t-tests at the 10% significance level. This experimental design provides a rigorous and statistically grounded comparison, isolating the contribution of the HyperSimplex formulation to generalization stability under varying batch regimes.

5.3 Results



For CIFAR-10, all seven configurations report positive mean accuracy differences, and all (100%) show statistically significant improvements at the 10% level ($p < 0.1$). For Fashion-MNIST, six of seven configurations ($\approx 86\%$) also achieve significance, with only the smallest batch size (128) falling above the 10% threshold. Across both datasets, therefore, 13 of 14 total comparisons ($\approx 93\%$) demonstrate statistically significant gains, indicating that the proposed loss systematically outperforms cross-entropy across a wide range of training conditions.

These findings confirm that the HyperSimplex loss maintains accuracy stability at smaller batch sizes while mitigating the degradation observed in cross-entropy as batch size increases. This supports its effectiveness as a smooth, geometry-consistent surrogate that enhances generalization and robustness in large-batch training regimes.

Table 1. Batch-wise Comparison of Cross-Entropy (CE) vs. HyperSimplex (HS) Losses on CIFAR-10 and Fashion-MNIST. The **highlighted** scores are statistically significant at a 10% level of significance.

	CIFAR-10					FashionMNIST				
Batch	**CE**	**HS (ours)**	**Δ**	**t-stat**	**p-val**	**CE**	**HS (ours)**	**Δ**	**t-stat**	**p-val**
128	0.8885	**0.8917**	0.0032	2.29	**0.084**	0.9456	0.9467	0.0011	1.31	0.262
256	0.8843	**0.8874**	0.0030	3.47	**0.026**	0.9434	**0.9473**	0.0040	4.34	**0.012**
512	0.8807	**0.8857**	0.0050	6.08	**<0.01**	0.9436	**0.9469**	0.0033	5.51	**<0.01**
1024	0.8776	**0.8821**	0.0045	9.78	**<0.01**	0.9413	**0.9453**	0.0040	6.67	**<0.01**
2048	0.8725	**0.8791**	0.0066	7.81	**<0.01**	0.9388	**0.9446**	0.0058	12.97	**<0.01**
4096	0.8659	**0.8750**	0.0090	13.14	**<0.01**	0.9371	**0.9439**	0.0069	9.68	**<0.01**
8192	0.8541	**0.8648**	0.0108	8.52	**<0.01**	0.9338	**0.9415**	0.0077	14.99	**<0.01**

5.4 Other Experiments: Cross-domain Validation

Additional GBRT results on tabular datasets for classification are reported in Appendix A, showing that the HyperSimplex loss also improves out-of-sample generalization beyond the neural settings studied in the main text.

6 Conclusion

We introduced the soft-binary-argmax@k, a differentiable projection onto the interior of the (n, k)-dimensional hypersimplex, and established its key properties—differentiability, order preservation, and efficient GPU computation. We showed how this operator integrates naturally into end-to-end learning systems, and used it to construct a surrogate to the zero–one loss for binary and multiclass settings, with statistically significant reductions in the generalization gap.

Owing to its close alignment with the true zero–one objective, the proposed HyperSimplex loss improves generalization under large-batch training. Additional cross-domain evaluations on tabular data further suggest that the benefits of the projection extend beyond neural models. Future work will explore applications to contrastive learning objectives and structured prediction.

A Appendix: Cross-Domain Tabular Results

Dataset	Higgs	Flight	KDD10	KDD12	Criteo	Avazu	KKBox	MovieLens
Cross Entropy	0.823	0.773	0.826	0.724	0.774	0.738	0.777	0.827
HyperSimplex	**0.846**	**0.778**	**0.849**	**0.729**	**0.796**	**0.741**	**0.797**	**0.828**

References

1. Agrawal, A., Amos, B., Barratt, S.T., Boyd, S.P., Diamond, S., Kolter, J.Z.: Differentiable convex optimization layers. CoRR **abs/1910.12430** (2019), http://arxiv.org/abs/1910.12430
2. Amos, B., Koltun, V., Kolter, J.Z.: The limited multi-label projection layer (2019), https://arxiv.org/abs/1906.08707
3. Bartlett, P.L., Jordan, M.I., McAuliffe, J.D.: Convexity, classification, and risk bounds. Journal of the American Statistical Association **101**(473), 138–156 (2006)
4. Best, M.J., Chakravarti, N.: Active set algorithms for isotonic regression: a unifying framework. Mathematical Programming **47**(1-3), 425–439 (1990). https://doi.org/10.1007/BF01580880
5. Blondel, M., Roulet, V.: The elements of differentiable programming (2024), https://arxiv.org/abs/2403.14606
6. Blondel, M., Teboul, O., Berthet, Q., Djolonga, J.: Fast differentiable sorting and ranking. In: International Conference on Machine Learning. pp. 950–959. PMLR (2020)

7. De Loera, J.A., Sturmfels, B., Thomas, R.R.: Gröbner bases and triangulations of the second hypersimplex. Combinatorica **15**(3), 409–424 (1995). https://doi.org/10.1007/BF01299745
8. Gneiting, T., Raftery, A.E.: Strictly proper scoring rules, prediction, and estimation. Journal of the American Statistical Association **102**(477), 359–378 (2007)
9. Gomez, C.: Fast Differentiable Projection Layers onto High-Dimensional Polytopes for Large-Scale Predictive Modeling. Ph.D. thesis, University of Central Florida (2025), https://stars.library.ucf.edu/etd2024/450
10. Gomez, C., Wang, P., Fu, Y.: Metric-agnostic learning-to-rank via boosting and rank approximation. In: 2023 IEEE International Conference on Data Mining (ICDM). pp. 1043–1048 (2023). https://doi.org/10.1109/ICDM58522.2023.00121
11. Gould, S., Fernando, B., Cherian, A., Anderson, P., Cruz, R.S., Guo, E.: On differentiating parameterized argmin and argmax problems with application to bi-level optimization. CoRR **abs/1607.05447** (2016), http://arxiv.org/abs/1607.05447
12. Hoffer, E., Hubara, I., Soudry, D.: Train longer, generalize better: Closing the generalization gap in large batch training of neural networks. In: Proceedings of the 31st Conference on Neural Information Processing Systems (NeurIPS). pp. 1731–1741 (2017)
13. Huber, P.J.: Robust estimation of a location parameter. The Annals of Mathematical Statistics **35**(1), 73–101 (1964)
14. Izmailov, P., Podoprikhin, D., Garipov, T., Vetrov, D., Wilson, A.G.: Averaging weights leads to wider optima and better generalization. In: Proceedings of the 34th Conference on Uncertainty in Artificial Intelligence (UAI). pp. 876–885 (2018), https://arxiv.org/abs/1803.05407
15. Keskar, N.S., Mudigere, D., Nocedal, J., Smelyanskiy, M., Tang, P.T.P.: On large-batch training for deep learning: Generalization gap and sharp minima. In: Proceedings of the 5th International Conference on Learning Representations (ICLR) (2017), https://openreview.net/forum?id=H1oyRlYgg
16. Krizhevsky, A.: Learning multiple layers of features from tiny images. Tech. Rep. TR-2009, University of Toronto (2009), https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf
17. Martins, A.F.T., Astudillo, R.F.: From softmax to sparsemax: A sparse model of attention and multi-label classification. CoRR **abs/1602.02068** (2016), http://arxiv.org/abs/1602.02068
18. Martins, A.F.T., Kreutzer, J.: Learning what‘s easy: Fully differentiable neural easy-first taggers. In: Palmer, M., Hwa, R., Riedel, S. (eds.) Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing. pp. 349–362. Association for Computational Linguistics, Copenhagen, Denmark (Sep 2017). https://doi.org/10.18653/v1/D17-1036, https://aclanthology.org/D17-1036/
19. Oyedotun, O.K., Papadopoulos, K., Aouada, D.: A new perspective for understanding generalization gap of deep neural networks trained with large batch sizes (2022), https://arxiv.org/abs/2210.12184
20. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S.: Pytorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems. pp. 8024–8035 (2019)
21. Xiao, H., Rasul, K., Vollgraf, R.: Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms. arXiv preprint arXiv:1708.07747 (2017), https://arxiv.org/abs/1708.07747